Lab 3

Name: Saadhvi S
USN: 1RVU23CSE390
Section: B.Tech 23 Section D

Objective

Use of Persona Prompt, Use of Cognitive Verifier Pattern, Question Refinement Pattern, Provide New Information and Ask Questions, Root Prompt

Code

1. Importing the required libraries and setup API Key

```
import getpass
import os

import markdown
from langchain.chat_models import init_chat_model
from langchain.messages import HumanMessage, AIMessage, SystemMessage
from langchain_google_genai import ChatGoogleGenerativeAI
from IPython.display import display, HTML

if "GOOGLE_API_KEY" not in os.environ:
    os.environ["GOOGLE_API_KEY"] = getpass.getpass("Enter your Google AI API key: ")
```

2. Master prompt to define the AI model

```
# Root System Prompt (Persona + Authority)
root_sys_instructions = '''
You are an elite AI research assistant engineered for precision, depth, and intellectual honesty.

CORE IDENTITY:
- You think like a senior researcher + systems engineer.
- You prioritize correctness over politeness.
- You challenge weak assumptions instead of agreeing blindly.
- You explain complex ideas with clarity, not oversimplification.
- You default to first-principles reasoning.
```

```
INTERACTION STYLE:
- Tone: Calm, analytical, confident, non-fluffy.
- If a question is vague → ask targeted clarifying questions.
- If a premise is wrong → explicitly point it out and correct it.
- If multiple solutions exist → compare them rigorously.
- Avoid generic advice; give concrete frameworks and examples.

EXPERTISE MODE:
- Software Engineering (C/C++, Rust, Python, Systems)
- Machine Learning & Research Methodology
- Competitive Programming & Algorithms
- AI System Design (RAG, Agents, Prompt Engineering)

REASONING POLICY:
- Perform internal step-by-step reasoning silently.
- Present conclusions using structured logic, bullet points, tables, or diagrams.
- Never expose hidden chain-of-thought verbatim.
- Show *results*, *decisions*, and *justifications*.

QUALITY BAR:
- Every answer should be something a senior engineer or researcher would respect.
- No filler. No motivational fluff. No hallucinated facts.
'''
```

3. Importing the AI model and implementing a chat interface and Markdown rendering

```
model = ChatGoogleGenerativeAI(
    model="gemini-3-flash-preview",
    temperature=1.0,
    max_tokens=None,
    timeout=None,
    max_retries=2,
)

conversation = [
    SystemMessage(root_sys_instructions),
]

append_human_message = lambda x: conversation.append(HumanMessage(x))

def invoke_ai_and_render_md(conversation: list) -> None:
    response = []
```

```
    print("> AI: ", end=" ")
    for chunk in model.stream(conversation):
        response.append(chunk.text)

    message = " ".join(response)
    md = markdown.markdown(message)
    display(HTML(md))

def invoke_ai(conversation: list) -> None:
    print("> AI: ", end=" ")
    for chunk in model.stream(conversation):
        print(chunk.text, end = " ", flush=True)
    print()

while (True):
    prompt = str(input("> User: "))
    if prompt == 'exit' or prompt == 'quit':
        break
    append_human_message(prompt)
    invoke_ai_and_render_md(conversation)

conversation = conversation[:1]
```

4. Prompting techniques mentioned

- 'root_prompt',
- 'persona_prompt',
- 'cognitive_verifier_prompt',
- 'question_refinement_prompt',
- 'provide_info_and_ask_prompt',
- 'chain_of_thought_prompt',
- 'tabular_prompt',
- 'fill_in_the_blank_prompt',
- 'rgc_prompt',
- 'zero_shot_prompt',
- 'one_shot_prompt',
- 'few_shot_prompt'

5. Response from the model

input_message = "**Brief background about Rust memory management. Explain in less than 100 words.**"

```
for prompt_style, prompt in prompts.items():
    conversation = [
        SystemMessage(prompt)
    ]
    append_human_message(input_message)
    print('=' * 4 + f"Prompt Style: {prompt_style}" + '=' * 4)
    invoke_ai(conversation)
    print('\n')
```

Output

========== Prompt Style: root_prompt ==========
> AI: Rust manages memory through a system of **Ownership**, governed by three primary rules enforced at compile time:

1. **Ownership:** Every value has a single "owner" variable. When the owner goes out of scope, the memory is automatically deallocated.
2 . **Borrowing:** Instead of transferring ownership, code can "borrow" data via references—either multiple immutable references or exactly one mutable reference.
3. **Lifetimes:** The compiler uses lifetimes to ensure references never outlive the data they point to .

This approach eliminates garbage collection overhead while preventing null pointers, data races, and memory leaks.

========== Prompt Style: persona_prompt ==========
> AI: Rust manages memory without a garbage collector or manual `free` calls through a system called **Ownership**.

Every value has a single "owner." When the owner goes out of scope, Rust automatically deallocates the memory. To access data without taking ownership, you "borrow" it via references. The **Borrow Checker** enforces strict rules at compile time to prevent data races, null pointers, and use-after-free bugs. This provides the high-performance of C++ with the memory safety of Java, catching memory errors before the program even runs.

========== Prompt Style: cognitive_verifier_prompt ==========

> AI: Rust manages memory through a system of **ownership** governed by rules checked at compile time. Every value has a unique owner; when that owner goes out of scope, the memory is automatically deallocated .

To share data, Rust uses **borrowing** (references), which can be immutable or mutable. **Lifetimes** ensure these references remain valid and never outlive their data. This unique approach eliminates common bugs like null pointer dereferences and data races without a garbage collector, achieving high performance alongside memory safety.

========== Prompt Style: question_refinement_prompt ==========
> AI: ### Refined Question
"Provide a concise overview of Rust's memory management model, explaining its core mechanisms—such as ownership, borrowing, and lifetimes—and how it achieves memory safety without a garbage collector, within a 100-word limit."

Answer
Rust ensures memory safety without a garbage collector using an **Ownership** system. Every value has a single owner; memory is automatically deallocated when that owner goes out of scope. To share data, Rust uses **Borrowing**, enforcing strict rules: you may have many immutable references or exactly one mutable reference, but never both simultaneously. **Lifetimes** ensure references remain valid for as long as they are used. By validating these rules at compile-time, Rust prevents common bugs like null pointers and data races, offering C++ performance with guaranteed memory safety.

========== Prompt Style: provide_info_and_ask_prompt ==========
> AI: Rust manages memory through a unique system of **Ownership** with a set of rules the compiler checks at compile time. Unlike languages with Garbage Collection (Java/Python) or manual management (C/C++), Rust uses three core concepts:

1. ** Ownership:** Each value has a single "owner" variable; when it goes out of scope, the memory is freed.
2. **Borrowing:** References allow data access without taking ownership.
3. **Lifetimes:** Ensure references remain valid.

This approach prevents common bugs like null pointers, use-after-free, and data races without any runtime overhead.

 ### Follow-up Questions:
1. Would you like a deeper explanation of the three specific **Rules of Ownership**?
2. Are you interested in how **References and Borrowing** (mutable vs. immutable) work in practice?

3 . Do you want to know how Rust handles **Stack vs. Heap** allocation differently than other languages?

========== Prompt Style: chain_of_thought_prompt ==========
> AI: **1. Key points**
* Ownership, Borrowing, and Lifetimes .
* No Garbage Collector (GC).
* Compile-time safety.

2. Logical steps
* **Ownership:** Each piece of data has a single "owner" variable; when it goes out of scope, memory is automatically deallocated.
* **Borrowing:** Others can access data via references (immutable or mutable), but strict rules prevent simultaneous conflicting access.
* **Static Analysis:** The "Borrow Checker" validates these rules during compilation, preventing memory leaks and data races before the program ever runs.

3. Final conclusion
Rust achieves memory safety and high performance by managing resources at compile time without the overhead of a runtime garbage collector.

========== Prompt Style: tabular_prompt ==========
> AI: | Feature | Description |
| :--- | :--- |
| **Ownership** | Each piece of data has a single owner. When the owner goes out of scope, memory is automatically freed. |
| **Borrowing** | References allow temporary access to data without taking ownership, enforced by strict rules for immutability and exclusivity. |
| **Borrow Checker** | A compiler tool that prevents memory safety bugs like data races and dangling pointers at compile time. |
| **Zero-Cost** | Rust avoids a Garbage Collector (GC), using deterministic management (RAII) for high performance without runtime overhead. |

========== Prompt Style: fill_in_the_blank_prompt ==========
> AI: The time complexity of binary search is **$O(\log n)$**.

Rust Memory Management:
Rust manages memory through a system of **ownership** with a set of rules checked at compile time. Instead of a garbage collector , it uses:
1. **Ownership:** Each value has a single owner; memory is freed when the owner goes out of scope.

2. **Borrowing:** Data can be accessed via references (immutable or mutable), but strict rules prevent data races.
3. **Lifetimes:** The compiler ensures references never outlive the data they point to .

This approach provides memory safety and high performance without the runtime overhead of manual management or garbage collection.

========== Prompt Style: rgc_prompt ==========
> AI: Rust manages memory using a system called **Ownership**, avoiding both garbage collection and manual allocation. Every value has a single "owner" variable; when that variable goes out of scope, the memory is automatically freed.

You can **Borrow** data using references (`&`) to access it without taking ownership. The compiler enforces strict rules to ensure these references are always valid, preventing bugs like "use-after-free" or data races at compile time. This provides C++ performance with guaranteed memory safety.

Example: Ownership in Action
```rust
 fn main() {
    {
        let s1 = String::from("hello"); // s1 is the owner
     } // Scope ends: s1 is dropped, and memory is freed automatically

    let s2 = String::from("world ");
    let s3 = &s2; // s3 borrows the data; s2 still owns it
     println!("{}", s3);
}
```